

Lab setup instructions and requirements

End Lab 00:59:20

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. Learn more.

Open Google Cloud console

Username

student-d81-d8f22b4bd0b6d

Password

RPxbc21n1kk

Project ID

qwiklabs-gcp-02-12ecff1

Getting Started with Firebase Genkit

🔗 Lab ⌚ 1 hour 💰 No cost 📈 Intermediate

★★★★☆ Rate Lab

📘 This lab may incorporate AI tools to support your learning.

Overview

Firebase Genkit is an open source framework that helps you build, deploy, and monitor production-ready AI-powered apps.



Firebase Genkit

Genkit is designed for app developers. It helps you to easily integrate powerful AI capabilities into your apps with familiar patterns and paradigms. The team behind Firebase builds it, and leverages their experience in building tools used by millions of developers worldwide. Genkit doesn't require a Firebase project and can be used in all Node.js environments, but it has integrations with Firebase and Google Cloud services through their own plugins.

Use **Genkit** to create apps that generate custom content, use semantic search, handle unstructured inputs, answer questions with your business data, autonomously make decisions, orchestrate tool calls, and much more.

Objective

This lab provides an introductory, hands-on experience with Firebase Genkit and the Gemini Flash foundation model. You create a simple application with two flows resolving two different prompts that accept input from the user.

You learn how to:

- Create a new Firebase Genkit project in your Cloud Shell environment.
- Create flows with Genkit and Gemini Flash.
- Create prompts with Dotprompt to provide structured input.
- Use the local Genkit developer UI.
- Explore flow and prompt traces in the Genkit development UI.

Setup and requirements

Before you click the **Start Lab** button

Note: Read these instructions.

Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources will be made available to you.

This Qwiklabs hands-on lab lets you do the lab activities yourself in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials that you use to sign in and access Google Cloud for the duration of the lab.

What you need

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).
- Time to complete the lab.

Note: If you already have your own personal Google Cloud account or project, do not use it for this lab.

Note: If you are using a Pixelbook, open an Incognito window to run this lab.

How to start your lab and sign in to the Console

1. Click the **Start Lab** button. If you need to pay for the lab, a pop-up opens for you to select your payment method. On the left is a panel populated with the temporary credentials that you must use for this lab.

Open Google Console

Lab instructions and tasks

0/100

Overview

Objective

Setup and requirements

Task 1. Set up your Genkit project

Task 2. Test your app in the Genkit Developer UI

Task 3. Add additional flows

Task 4. Create strongly typed prompts

Congratulations!

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. [Learn more.](#)

Username
google12727832_student@qwiklabs.net

Password
k58CZKsXKZ

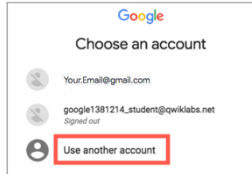
GCP Project ID
qwiklabs-gcp-4f7fecac8667e457

[New to labs? View our introductory video!](#)

- Copy the username, and then click **Open Google Console**. The lab spins up resources, and then opens another tab that shows the **Choose an account** page.

Note: Open the tabs in separate windows, side-by-side.

- On the Choose an account page, click **Use Another Account**. The Sign in page opens.



- Paste the username that you copied from the Connection Details panel. Then copy and paste the password.

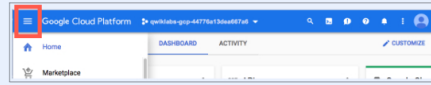
Note: You must use the credentials from the Connection Details panel. Do not use your Google Cloud Skills Boost credentials. If you have your own Google Cloud account, do not use it for this lab (avoids incurring charges).

- Click through the subsequent pages:

- Accept the terms and conditions.
- Do not add recovery options or two-factor authentication (because this is a temporary account).
- Do not sign up for free trials.

After a few moments, the Cloud console opens in this tab.

Note: You can view the menu with a list of Google Cloud Products and Services by clicking the **Navigation menu** at the top-left.



Activate Google Cloud Shell

Google Cloud Shell is a virtual machine that is loaded with development tools. It offers a persistent 5GB home directory and runs on the Google Cloud.

Google Cloud Shell provides command-line access to your Google Cloud resources.

- In Cloud console, on the top right toolbar, click the Open Cloud Shell button.



- Click **Continue**.

It takes a few moments to provision and connect to the environment. When you are connected, you are already authenticated, and the project is set to your *PROJECT_ID*. For example:



gcloud is the command-line tool for Google Cloud. It comes pre-installed on Cloud Shell and supports tab-completion.

- You can list the active account name with this command:

```
gcloud auth list
```

Output:

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
</mydomain></myaccount>
```

Example output:

```
Credentialed accounts:
- google1623927_student@qwiklabs.net
```

- You can list the project ID with this command:

```
gcloud config list project
```

Output:

```
[core]
project = <project_id>
</project_id>
```

Example output:

```
[core]
project = qwiklabs-gcp-44776a13dea667a6
```

Note: Full documentation of **gcloud** is available in the [gcloud CLI overview guide](#).

Task 1. Set up your Genkit project

In this task, you set up a node application to use Firebase Genkit, and configure it to access the Gemini model in Google Cloud's Vertex AI.

Initialize your Genkit environment

1. To initialize your environment for Google Cloud, run the following commands:

```
export GCP_PROJECT=qwiklabs-gcp-82-12ecff12336f
export GCP_LOCATION=us-central1
```

2. To authenticate to Google Cloud and set up credentials for your project and user, run the following command:

```
gcloud auth application-default login
```

3. When prompted, type Y, and press ENTER.
 4. To launch the Google Cloud sign-in flow, press Control (for Windows and Linux) or Command (for MacOS), and click the link in the terminal.
 5. Click the student email address.
 6. When you're prompted to continue, click **Continue**.
 7. To let the Google Cloud SDK access your Google Account and agree to the terms, click **Allow**.
- Your verification code is displayed in the browser tab.
8. Click **Copy**.
 9. Back in the terminal, where it says **Enter authorization code**, paste the code and press ENTER.

You are now authenticated to Google Cloud.

Set up your application

1. Create a directory for your project, and initialize a new Node project:

```
mkdir genkit-intro && cd genkit-intro
npm init -y
```

This command creates a `package.json` file in the `genkit-intro` directory.

2. To install the Genkit CLI, run the following command:

```
npm install -D genkit-cli@1.4.0
```

3. To install the core Genkit packages and dependencies for your app, run the following command:

```
npm install genkit@1.4.0 --save
npm install @genkit-ai/vertexai@1.4.0 @genkit-ai/google-cloud@1.4.0 @genkit-ai/express@1.4.0 --save
```

Create the Genkit app code

1. Create the application source folder and main file:

```
mkdir src && touch src/index.ts
```

2. From the **Cloud Shell** menu, click **Open Editor**.
3. Open the `genkit-intro/package.json` file, and review all the dependencies that were added with Genkit.

The dependency list should look similar to this:

```
{
  "dependencies": {
    "@genkit-ai/express": "^1.4.0",
    "@genkit-ai/google-cloud": "^1.4.0",
    "@genkit-ai/vertexai": "^1.4.0",
    "genkit": "^1.4.0"
  }
}
```

4. Go to the `src` folder, and open the `src/index.ts` file. Add the following import

library references to the `index.ts` file:

```
import { z, genkit } from 'genkit';
import { vertexAI } from '@genkit-ai/vertexai';
import { gemini20Flash001 } from '@genkit-ai/vertexai';
import { logger } from 'genkit/logging';
import { enableGoogleCloudTelemetry } from '@genkit-ai/google-cloud';
import { startFlowServer } from '@genkit-ai/express';
```

5. After the import statements, add code to initialize and configure the genkit instance with the required plugin:

```
const ai = genkit({
  plugins: [
    vertexAI({ location: 'us-central1' }),
  ]
});

logger.setLogLevel('debug');
enableGoogleCloudTelemetry();
```

The Vertex AI plugin provides access to the Gemini models. The initialization code also sets the log level to debug for troubleshooting, and enables tracing and metrics for monitoring.

6. Define a flow for your application:

```
export const menuSuggestionFlow = ai.defineFlow(
{
  name: 'menuSuggestionFlow',
  inputSchema: z.string(),
  outputSchema: z.string(),
},
async (subject) => {
  // Construct a request and send it to the model API.
  const llmResponse = await ai.generate({
    prompt: 'Suggest an item for the menu of a',
    $(subject) themed restaurant',
    model: gemini20Flash001,
    config: {
      temperature: 1,
    },
  });
  // Handle the response from the model API. In this
  // sample, we just convert
  // it to a string, but more complicated flows might
  // coerce the response into
  // structured output or chain the response into another
  // LLM call, etc.
  return llmResponse.text;
});
```

A **flow** to suggest menu items is created. The **subject** of the menu is passed in as a parameter and inserted into the prompt to the model. The model used is `gemini20Flash001` with a **temperature** configuration setting of 1.

7. Add the following code to start the flow server, which exposes your flow as an HTTP endpoint:

```
startFlowServer({
  flows: [menuSuggestionFlow],
});
```

8. To return to the command line, click **Open Terminal**.

9. To copy the `package.json` and `index.ts` files to a Cloud Storage Bucket, run the following commands:

```
gcloud storage cp ~/genkit-intro/package.json
gs://wikilabs-gcp-02-12ecff12336f
gcloud storage cp ~/genkit-intro/src/index.ts
gs://wikilabs-gcp-02-12ecff12336f
```

Click **Check my progress** to verify the objectives.



Setup your Genkit Project

Check my progress

Task 2. Test your app in the Genkit Developer UI

The Genkit developer UI is a local web app that you can use to interact with models, retrievers, flows, and other actions in your Genkit project.

In this task, you use the Genkit developer UI to explore AI workflows, models, and parameters, and analyze traces for your app.

Launch the Genkit developer UI

1. To launch the Genkit developer UI from your Cloud Shell terminal, run:

```
cd ~/genkit-intro
npx genkit start -- npx tsx src/index.ts
```

2. Press ENTER to continue.

Wait for the command to return the **Genkit Developer UI** URL in the output, and indicate the flow server is running, before continuing to the next step.

3. In the **Web Preview** menu () , click **Change port**.
 4. For the **Port Number**, type **4000**, and click **Change and Preview**.
- This opens the Genkit developer UI in a separate tab in your browser.

Explore the UI

1. To explore your predefined workflows, in the left panel click **Flows**, and then click **menuSuggestionsFlow**.
 2. For **Input JSON**, type **"Spanish"** (include the quotation marks), and click **Run**.
- The flow executes, and the model response is displayed in the UI.
3. To analyze the execution time and metadata associated with the flow, scroll to the bottom, and click **View trace**.
- The trace shows the overall time taken by **menuSuggestionFlow**, and the subset of that time taken by the **vertex/gemini-2.0-flash-001** model.
4. Click **menuSuggestionFlow**, and click **Open in Flow Runner**.
- The flow can be run with another input. Experiment with a different string value in the input JSON field, for example, **"Indian"**.
5. In the left panel, click **Models**. In the **Config** tab on the right, select **vertexai/gemini-2.0-flash-001**.
- Several models available in Vertex AI are listed, including the Flash model used by your application. The Gemini 2.0 Flash 001 model is selected.
6. In the **Config** pane, modify the **Temperature**.
 7. For **Enter a message**, set the input to:

Suggest an item for the menu of a Greek themed restaurant 

8. Click **Run**.
- The new prompt is submitted to the Gemini 2.0 Flash 001 model. A Greek menu item is returned.
9. Click **View Trace**.
- In this case, the trace shows the input to and output from the Flash model.
10. To review traces of all the executed flows and models, in the left panel, click **Traces**.
- You can click on any trace to review the input, output, and steps for that run.
11. To exit the Genkit UI, close your browser tab. Then, in Cloud Shell, type CTRL+C to stop the application.

Task 3. Add additional flows

In this task, you add additional flows to your project, explore them in the UI, and leverage the Genkit CLI to execute the flows and stream the responses.

Create a new flow

1. Open the Cloud Shell editor, and locate and open the `src/index.ts` file.
2. Insert the new flow `jokeFlow` above the line that starts the flow server.

```
export const jokeFlow = ai.defineFlow({
  name: 'jokeFlow',
  inputSchema: z.string(),
  outputSchema: z.string(),
},
async (subject) => {
  const llmResponse = await ai.generate({
    prompt: 'Tell me a joke about ${subject}',
    model: gemini20Flash001,
    config: {
      temperature: 1,
    },
  });
  return llmResponse.text;
});
```

This flow generates a joke based on a user-suggested topic using the generate function and the `gemini20Flash001` model.

3. Update the list of flows to be started:

```
startFlowServer({
  flows: [menuSuggestionFlow, jokeFlow],
});
```

Run the flow in the Genkit Developer UI

1. After your changes are saved, open the Cloud Shell terminal, and start the Genkit Developer UI:

```
cd ~/genkit-intro
npx genkit start -- npx tsx src/index.ts
```

```
npm run dev
```

Wait for the UI and flow server to start before proceeding to the next step.

2. To access the Genkit Developer UI in your browser, use the **Web Preview** option, and set the port to **4000**.
3. In the UI, click **Flows**, and then click **jokeFlow**.
4. Provide a topic for the joke. For **Input JSON**, between the quotes, type **cats**, and click **Run**.

A joke on the topic of cats is generated by the model and returned in the response.

Use the Genkit CLI

1. You can use the Genkit CLI to execute flows directly. In Cloud Shell, open a new Cloud Shell terminal, and run the **menuSuggestionFlow**:

```
cd ~/genkit-intro
npx genkit flow:run menuSuggestionFlow "French" -s | tee
-a output.txt
```

View the response from the model.

2. Exit the UI. In the original Cloud Shell terminal type CTRL+C to stop the Genkit application.
3. Finally, copy the `output.txt` and `src/index.ts` files to a Cloud Storage bucket:

```
gcloud storage cp -r ~/genkit-intro/output.txt
gs://qwiklabs-gcp-02-12ecff12336f
gcloud storage cp -r ~/genkit-intro/src/index.ts
gs://qwiklabs-gcp-02-12ecff12336f
```

Click **Check my progress** to verify the objectives.

☐

Add additional flows

Check my progress

Task 4. Create strongly typed prompts

In this task, you create a flow that receives structured input that contains a customer name, the current time, and data from a previous order. The flow returns a recommendation based on the input.

You learn to structure the input as an object, and define the prompt using `Dotprompt`. This structure is more composable and will help you organize more complex applications.

Create the flow and related objects

1. Open the `src/index.ts` file in the Cloud Shell editor.
2. Below the last flow `jokeFlow`, and above the `startFlowServer()` function call, define the input schema that contains the customer name, current time, and previous order.

```
const CustomerTimeAndHistorySchema = z.object({
  customerName: z.string(),
  currentTime: z.string(),
  previousOrder: z.string(),
});
```

3. Below the `CustomerTimeAndHistorySchema` schema definition, define the prompt object that the flow will use. The prompt contains the input schema, the model to use, and the text prompt.

```
const greetingWithHistoryPrompt = ai.definePrompt(
{
  name: 'greetingWithHistory',
  model: gemini20Flash001,
  input: { schema: CustomerTimeAndHistorySchema },
  output: {
    format: 'text',
  },
},
{
  {{role "user"}}
  Hi, my name is {{customerName}}. The time is
  {{currentTime}}. Who are you?
  {{role "model"}}
  I am Barb, a barista at this nice underwater-themed coffee
  shop called Krabby Koffee.
  I know pretty much everything there is to know about
  coffee,
  and I can cheerfully recommend delicious coffee drinks to
  you based on whatever you like.
  {{role "user"}}
  Great. Last time I had {{previousOrder}}.
  I want you to greet me in one sentence, and recommend a
  drink.
  .
});
```

By default, `Dotprompt` constructs a single message with a 'user' role. Some prompts are best expressed as a combination of multiple messages. The `{{role}}` helper provides a simple way to construct multi-message prompts.

4. Below the `greetingWithHistoryPrompt` prompt definition, define the flow that uses the input schema and the defined prompt.

```
export const greetingFlow = ai.defineFlow(  
  {  
    name: 'greetingFlow',  
    inputSchema: CustomerTimeAndHistorySchema,  
    outputSchema: z.string(),  
  },  
  async (input) => {  
    const response = await greetingWithHistoryPrompt({  
      customerName: input.customerName,  
      currentTime: input.currentTime,  
      previousOrder: input.previousOrder  
    });  
    return response.text;  
  }  
);
```

5. Update the `startFlowServer` call with the list of flows to be started:

```
startFlowServer({  
  flows: [menuSuggestionFlow, jokeFlow, greetingFlow],  
});
```

Test the flow

1. After your changes are saved, open the Cloud Shell terminal, and start the Genkit Developer UI:

```
cd ~/genkit-intro  
npm run genkit start -- npx tsx src/index.ts
```

Wait for the Genkit Developer UI and the flow server to start before proceeding to the next step.

2. To access the Genkit Developer UI in your browser, use the **Web Preview** option, and set the port to **4000**.
3. In the UI, click **Prompts**, and click the **greetingWithHistory** prompt.
4. Provide the **Input (JSON)** below:

```
{  
  "customerName": "Hugo",  
  "currentTime": "12pm",  
  "previousOrder": "Coffee"  
}
```

The user and model prompts are updated in the UI.

5. To execute the prompt and view the response, click **Run**.
6. In the left panel, click **Flows**, and then click the **greetingFlow** flow.
7. Provide the same **Input (JSON)** below:

```
{  
  "customerName": "Hugo",  
  "currentTime": "12pm",  
  "previousOrder": "Coffee"  
}
```

8. To execute the flow and view the response, click **Run**.
9. To analyze the flow's execution details, timing and metadata, after the flow executes, click **View Trace**.
10. Exit the UI, and type CTRL+C in your Cloud Shell terminal to stop the Genkit application.
11. Copy the `src/index.ts` file to the Cloud Storage bucket.

```
gcloud storage cp -r ~/genkit-intro/src/index.ts  
gs://qwiklabs-gcp-02-12ecff12336f
```

Click **Check my progress** to verify the objective.



Define strongly typed prompts

Check my progress

Congratulations!

Congratulations! You have successfully created your first Genkit project. In this lab, you defined several flows, interacted with structured input, and managed prompts using Dotprompt. Additionally, you tested the application through both the Genkit development UI and the CLI interface.

Next steps / Learn more

For more information about the topics discussed in this lab, view these links:

- [Firebase Genkit](#)
- [Managing prompts with Dotprompt](#)

Manual Last Updated March 24, 2025

Lab Last Tested February 20, 2025

